

Getting into the specification and scope changes, at the beginning of this project, the plan was ambitious but exciting: train deep learning models that could detect and classify both fish and prawns from underwater images, and ultimately build a real-time detection system using the best-performing model. The specification clearly outlined tasks such as dataset collection for both fish and prawns, model development, evaluation, and the creation of an end-to-end pipeline that could potentially integrate with underwater robotic systems. I understood this would be challenging, but I initially believed that with enough time, it would be achievable.

One of the first major roadblocks I encountered was the dataset. I spent about a week scouring the internet, academic archives, GitHub repositories, and even reaching out on forums, trying to find a publicly available dataset that included both fish and prawns. While there were a few prawn-specific papers, the datasets themselves were either not available, poorly labeled, or too limited in size and diversity. After several frustrating days, I realised that continuing this search would eat into the time I needed to actually start the technical side of the project. I scheduled a meeting with my supervisor to explain the situation, and he was understanding. With his support, I narrowed the focus of the project to fish detection only, using the Fish4Knowledge dataset. It was a compromise, but it allowed me to move forward rather than stall indefinitely.

Another significant deviation from the original plan was the system development component. The specification had stated that the project should include an end-to-end detection system, potentially using microservices and being deployable in real-time. At first, I assumed this was just a way of saying “build a working model.” As I got deeper into the project, I realised this would require substantial software engineering work ranging from backend integration, service containerization, and possibly building a user-facing interface or deployment pipeline. This was far beyond what I had experience in, and I struggled even to visualise what the end product would look like. Again, I raised the issue with my supervisor, and we agreed that the project should instead focus on model development, training, and evaluation — areas where I could contribute meaningfully within the timeframe and with the skills I had.

These changes significantly altered the direction and scope of the project, and they came with a certain amount of guilt and anxiety. It felt at times like I was failing to meet the brief. But looking back now, I think the decisions were justified. The project remained technically rich, involved a lot of coding, debugging, experimentation, and writing. I didn’t take the easy way out — I just redirected the effort toward areas where I could actually grow and succeed.

The experience taught me that adjusting the project scope is not the same as failing it. Recognising when something is not feasible and having the initiative to propose changes is a skill in itself. It also taught me to communicate more proactively whereby if I had raised my dataset and system development concerns earlier, I might have avoided some of the stress that came later.

Ultimately, the core learning outcomes of the project — model design, dataset preparation, evaluation, and critical comparison — were met. I gained hands-on experience with convolutional and transformer-based architectures, learned how to preprocess real-world datasets for multiple model types, and wrote detailed evaluation logic to compare object detection and classification fairly. While the end product was not an integrated system, it was still a meaningful technical contribution and a valuable learning experience.

As for the self-reflection part, at the start of this project, I had a Gantt chart laid out from the end of November to April, clearly outlining when each part of the work should be completed. In theory, that structure should have been enough to keep me on track. In practice, I procrastinated. Other responsibilities and personal distractions got in the way, and I let the earlier months slide by without engaging deeply with the project. As a result, a significant portion of the workload was left until late February, which meant compressing what should have been a five-month process into a few high-pressure weeks.

When I finally sat down to work on the project properly, I underestimated how long certain tasks would take — especially when preparing the Fish4Knowledge dataset. I initially assumed the data was ready to use, but it turned out I needed to generate bounding boxes from segmentation masks, clean and restructure the folders, and manually verify annotation quality. That delay created a domino effect that pushed model training and evaluation later than I had planned. I also encountered frustrating errors, particularly with Swin Transformer. Training results were inconsistent, and I struggled with increasing loss and stagnant accuracy. At one point I thought I had done something fundamentally wrong, only to discover the problem was related to data augmentation and learning rate settings. Those setbacks were difficult emotionally as it was disheartening to put in hours of work and still get poor results. But they also taught me that trial and error is part of deep learning, and that it's okay to go back and reconfigure even after progress has been made.

Another learning curve came with evaluation. YOLOv8 didn't provide F1-score or accuracy in the way classification models do, and I had to manually compute and log values for fair comparison. It took me time to realise that detection and classification models require different metric interpretations which was something I hadn't considered when planning the comparison. There were also practical issues. I struggled with managing Python environments, CUDA dependencies, and symbolic link warnings on Windows. At one point, I installed the wrong version of a package and broke my environment completely, which meant having to rebuild it while under time pressure. These moments, though frustrating, pushed me to improve my technical self-sufficiency.

Looking back, I recognise that my biggest mistake was assuming I'd have more time later. I've always worked well under pressure, but this project showed me that deep learning is not something you can "cram." Training takes hours, evaluation takes thought, and writing takes clarity. I do think I've developed strong research resilience through this — I now know how to troubleshoot PyTorch issues, adapt model scripts, and interpret result curves with more confidence. I've also improved my academic writing by learning to balance technical explanation with accessibility, especially when preparing the comparative analysis and executive summary.

One of the most valuable aspects of the project was seeing how all parts — data, model, metrics, results — needed to align. This wasn't just about model accuracy; it was about building a coherent pipeline, understanding what each component contributes, and being able to explain it to someone else. I also saw how reflection plays a role in technical work as not just learning how to fix bugs, but understanding what led to them in the first place. If I were to do this again, I would start earlier and I genuinely mean that. I would set internal deadlines that fall well before the actual ones and treat them seriously. I would also document my model runs more consistently (with config files or run logs) and keep backups of working environments. These habits might have saved me from repeating mistakes or losing progress when errors occurred.

In the end, I persevered and completed the project, and I'm proud of how it turned out. But more importantly, I've come away with hard-earned skills, a clearer sense of how I work under pressure, and a better appreciation for planning, pacing, and technical depth. This wasn't a perfect journey, but it was a meaningful one.